

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“EKSPANSI PRODUK KEUANGAN (INHERITANCE)”

disusun Oleh:
Muhammad Fedora Argadyaksa
2511533016

Dosen Pengampu:
Nurfiah, S.ST, M.Kom
Asisten Praktikum:
Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

1. Langkah Kerja Praktikum

a. Langkah pertama yaitu, buka aplikasi yang dibutuhkan untuk praktikum (Eclipse) Lalu buat package pekan 4 dan berisi class Rekening, Main, Transaksi yang menggunakan kode program pada pekan sebelumnya dan juga terdapat class baru yang diberikan pada modul ini.

b. Langkah kedua, membuat kode program yang terdapat penambahan atau perubahan di dalam modul praktikum :

1. Class Rekening :

```
1. package Pekan_4;
2.
3. import java.util.ArrayList;
4.
5. public class Rekening {
6.     // Bagian atas dari class Rekening.java
7.     private String nomorRekening;
8.     private String namaPemilik;
9.     private String pin; // data sensitif
10.
11.     // Gunakan protected agar Subclass bisa mengaksesnya langsung
12.     protected double saldo;
13.     protected ArrayList<Transaksi> riwayatTransaksi;
14.
15.     // Isi konstruktor dan metode lainnya TETAP SAMA seperti
    Modul 3
```

Kode ini merupakan bagian awal dari class **Rekening** dalam package **Pekan_4**, yang merupakan pengembangan lebih lanjut dari konsep enkapsulasi pada **Pekan_3**, dengan tambahan persiapan untuk konsep **pewarisan (inheritance)**. Atribut **nomorRekening**, **namaPemilik**, dan **pin** tetap dijadikan **private**, karena ketiganya bersifat data pribadi dan sensitif yang tidak perlu maupun tidak boleh diakses langsung oleh class lain, termasuk subclass sekalipun.

Namun, atribut **saldo** dan **riwayatTransaksi** kini diubah modifier aksesnya menjadi **protected**, bukan lagi **private** seperti pada **Pekan_3**. Perubahan ini bertujuan agar kedua atribut tersebut dapat **diakses langsung**

oleh subclass (class turunan) dari Rekening, tanpa harus melalui getter/setter, karena nantinya subclass mungkin memerlukan akses langsung terhadap saldo maupun riwayat transaksi untuk keperluan logika bisnis tambahan yang spesifik pada turunannya. Sementara itu, bagian constructor dan method lainnya dinyatakan tetap sama seperti pada Modul 3 (Pekan_3), sehingga logika validasi PIN, pembuatan objek, serta method-method seperti setorTunai(), tarikTunai(), otentikasi(), dan lainnya tidak mengalami perubahan.

2. Class Main

Untuk kelas Main tidak ada perubahan dari modul, tetapi terdapat perubahan pada bagian tugas/latihan praktikum. Jadi kelas main masih sama seperti pada pekan sebelumnya.

3. Class Transaksi

Untuk kelas Transaksi tidak ada perubahan dari modul, Jadi kelas main masih sama seperti pada pekan sebelumnya.

4. Class RekeningTabungan

```
1. package Pekan_4;
2.
3. public class RekeningTabungan extends Rekening {
4.
5.     // Atribut spesfik yang hanya dimiliki oleh Tabungan
6.     private double sukuBunga;
7.
8.
9.     // Contructor Subclass
10.    public RekeningTabungan (String nomor, String nama, double
        saldoAwal, String pinAwal, double sukuBunga) {
11.        // super() memanggil constructor kelas induk (Rekening),
        WAJIB berada di baris pertama
12.        super(nomor, nama, saldoAwal, pinAwal);
13.        this.sukuBunga = sukuBunga;
14.    }
15.
16.    public void tambahBungaAkhirBulan() {
17.        // Menghitung Bunga
18.        // Mengapa bisa mengakses saldo secara langsung dari
        class RekeningTabungan ?
```

```

19.         double nominalBunga = saldo * (sukuBunga / 100);
20.         saldo += nominalBunga;
21.
22.         // Mencatat riwayat transaksi
23.         String idTrx = "TRX-B" + System.currentTimeMillis();
24.         riwayatTransaksi.add(new Transaksi(idTrx, "Bunga",
nominalBunga));
25.
26.         System.out.println("Bunga " + sukuBunga + "% berhasil
ditambahkan : Rp" + nominalBunga);
27.
28.     }
29.
30. }

```

Kode ini merupakan class RekeningTabungan dalam package Pekan_4, yang merupakan subclass (class turunan) dari class Rekening menggunakan keyword extends, sehingga menerapkan konsep pewarisan (inheritance) dalam OOP. Class ini memiliki atribut tambahan yang spesifik hanya dimiliki oleh rekening tabungan, yaitu sukuBunga bertipe double dan bersifat private, karena hanya relevan dan digunakan di dalam class ini sendiri.

Pada constructor RekeningTabungan, parameter yang diterima meliputi nomor, nama, saldoAwal, pinAwal, dan sukuBunga. Di baris pertama constructor, dipanggil super(nomor, nama, saldoAwal, pinAwal), yang berfungsi untuk memanggil constructor milik class induk (Rekening) guna menginisialisasi atribut-atribut yang diwariskan dari Rekening. Pemanggilan super() ini wajib diletakkan di baris pertama constructor subclass, karena proses inisialisasi objek induk harus selesai terlebih dahulu sebelum subclass menambahkan inisialisasi miliknya sendiri, dalam hal ini adalah atribut sukuBunga.

Selanjutnya terdapat method tambahBungaAkhirBulan(), yang berfungsi untuk menghitung dan menambahkan bunga ke saldo rekening pada akhir bulan. Nominal bunga dihitung dari saldo * (sukuBunga / 100), kemudian hasilnya ditambahkan langsung ke saldo. Perlu diperhatikan bahwa atribut saldo dapat diakses secara langsung di class ini (tanpa getter/setter) karena pada class Rekening, saldo dideklarasikan dengan modifier protected, bukan private, sehingga dapat diwariskan dan diakses langsung oleh subclass-

nya, termasuk RekeningTabungan ini. Hal yang sama berlaku untuk atribut riwayatTransaksi, yang juga bersifat protected, sehingga method ini bisa langsung memanggil riwayatTransaksi.add() untuk mencatat transaksi baru bertipe "Bunga" dengan id transaksi unik berdasarkan waktu sistem. Terakhir, program menampilkan pesan konfirmasi bahwa bunga berhasil ditambahkan beserta nominalnya.

5. Class RekeningGiro

```
1. package Pekan_4;
2.
3. public class RekeningGiro extends Rekening{
4.     private double batasOverdraft;
5.
6.     public RekeningGiro (String nomor, String nama, double
saldoAwal, String pinAwal, double batasOverdraft) {
7.         // Memanggil inisialisasi dasar dari Superclass
8.         super(nomor, nama, saldoAwal, pinAwal);
9.         this.batasOverdraft = batasOverdraft;
10.    }
11.
12.    // Getter khusus Giro
13.    public double getBatasOverdraft() {
14.        return batasOverdraft;
15.    }
16.
17.    // Catatan Penarikan hingga limit overdraft akan kita
selesaikan di modul 5
18.
19. }
```

Kode ini merupakan class RekeningGiro dalam package Pekan_4, yang juga merupakan subclass dari class Rekening menggunakan extends, sama seperti RekeningTabungan sebelumnya, sehingga menerapkan konsep pewarisan (inheritance). Class ini memiliki atribut tambahan yang spesifik untuk rekening giro, yaitu batasOverdraft bertipe double dan bersifat private, yang berfungsi untuk menyimpan batas maksimal penarikan yang diperbolehkan meski saldo mencapai minus (overdraft).

Pada constructor RekeningGiro, parameter yang diterima meliputi nomor, nama, saldoAwal, pinAwal, dan batasOverdraft. Sama seperti pada RekeningTabungan, baris pertama constructor memanggil super(nomor,

nama, saldoAwal, pinAwal) untuk menjalankan inisialisasi dasar dari superclass (Rekening), sebelum kemudian menginisialisasi atribut batasOverdraft yang khusus dimiliki oleh RekeningGiro.

Selanjutnya disediakan method `getBatasOverdraft()` sebagai getter khusus untuk membaca nilai batasOverdraft dari luar class, mengikuti prinsip enkapsulasi yang tetap diterapkan meskipun class ini adalah subclass. Terakhir, terdapat komentar yang menandakan bahwa logika penarikan dana hingga melewati saldo (memanfaatkan batas overdraft) belum diimplementasikan pada tahap ini dan direncanakan akan diselesaikan pada modul berikutnya (Modul 5), sehingga class ini untuk sementara hanya menyimpan data batas overdraft tanpa mengubah perilaku transaksi yang diwariskan dari Rekening.

2. Tugas/Latihan Praktikum

1. Pembuatan Multi-Produk

Modifikasi case 1: Buka Rekening Baru di Main.java.

- a. Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
- b. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
- c. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
- d. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

2. Simulasi Waktu Akhir Bulan

- a. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
- b. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator `instanceof`.

- c. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- d. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

Jawaban :

1. Kode program class Main :

```
switch (pilihan) {
case 1 :
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal : ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan PIN : ");
    String pin = input.nextLine();
    System.out.print("Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :");
    int pil = input.nextInt();
    if (pil == 1) {
        System.out.print("Masukkan Suku Bunga (%): ");
        double sukuBunga = input.nextDouble();
        akunAktif = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    } else if (pil == 2) {
        System.out.print("Masukkan Batas Overdraft: ");
        double batasOverdraft = input.nextDouble();
        akunAktif = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
    } else {
        System.out.println("Pilihan tidak valid");
    }

    if (akunAktif != null) {
        rekeningAktif.add(akunAktif);
        System.out.println("Rekening berhasil ditambahkan. Total rekening: " + rekeningAktif.size());
    }
    break;
```

Bagian awal (input nomor rekening, nama, saldo awal, dan PIN) tidak mengalami perubahan dan tetap sama seperti Pekan_3. Penambahan baru dimulai setelah input PIN, yaitu program kini menampilkan pilihan jenis produk rekening kepada pengguna melalui `System.out.print("Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :")`, yang dibaca menggunakan `input.nextInt()` dan disimpan ke variabel `pil`.

Berdasarkan pilihan tersebut, ditambahkan struktur `if-else` untuk menentukan jenis objek yang akan dibuat, memanfaatkan konsep pewarisan (inheritance) dari Pekan_4: jika `pil == 1`, program meminta input tambahan berupa suku bunga, lalu membuat objek dari subclass `RekeningTabungan`; jika `pil == 2`, program meminta input batas overdraft, lalu membuat objek dari subclass `RekeningGiro`; dan jika input tidak sesuai (bukan 1 atau 2), program

menampilkan pesan "Pilihan tidak valid" tanpa membuat objek apa pun, sehingga akunAktif tetap bernilai null.

Penambahan penting lainnya adalah adanya pengecekan if (akunAktif != null) sebelum proses rekeningAktif.add(akunAktif) dan penampilan pesan sukses dijalankan. Penambahan ini diperlukan sebagai antisipasi terhadap kemungkinan pilihan produk yang tidak valid, sehingga program tidak keliru menambahkan data kosong (null) ke dalam list rekeningAktif atau menampilkan pesan sukses yang salah, sesuatu yang tidak perlu ada pada Pekan_3 karena di sana hanya ada satu jenis objek Rekening yang pasti langsung dibuat tanpa percabangan pilihan produk.

Output :

```
=== SISTEN PERBANKAN MINI ===  
  
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
Pilih Menu : 1  
Masukkan No Rekening: 0000  
Masukkan Nama Pemilik : Fedo  
Masukkan Saldo Awal : 200000  
Masukkan PIN : 000000  
Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :1  
Masukkan Suku Bunga (%): 20  
Rekening berhasil ditambahkan. Total rekening: 1
```

Jika salah menginput pilihan :

```
=== SISTEN PERBANKAN MINI ===  
  
Menu Utama :  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
Pilih Menu : 1  
Masukkan No Rekening: 000  
Masukkan Nama Pemilik : fedo  
Masukkan Saldo Awal : 200000  
Masukkan PIN : 000000  
Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :3  
Pilihan tidak valid
```

2. Kode program class Main :

```
case 7 :
    for (Rekening tabungan : rekeningAktif) {
        System.out.println("No: " + tabungan.getNomorRekening() + ", Nama: " + tabungan.getNamaPemilik());

        if (tabungan instanceof RekeningTabungan) {
            RekeningTabungan rekeningTabungan = (RekeningTabungan) tabungan; // downcasting
            rekeningTabungan.tambahBungaAkhirBulan();
        } else {
            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan");
        }
    }
    break;
```

Kode ini merupakan potongan **case 7**, yang berfungsi untuk menjalankan fitur penambahan bunga akhir bulan pada seluruh rekening yang tersimpan, dengan memanfaatkan konsep **polimorfisme** dan **downcasting**. Program menggunakan perulangan **for-each** `for (Rekening tabungan : rekeningAktif)` untuk mengakses setiap objek rekening yang ada di dalam list `rekeningAktif` satu per satu, meskipun list tersebut sebenarnya bisa berisi campuran objek `RekeningTabungan` maupun `RekeningGiro`, karena keduanya merupakan turunan dari class `Rekening`. Pada setiap iterasi, program menampilkan informasi nomor rekening dan nama pemilik menggunakan method `getNomorRekening()` dan `getNamaPemilik()` yang diwarisi dari class `Rekening`.

Selanjutnya, dilakukan pengecekan menggunakan `instanceof` melalui kondisi `if (tabungan instanceof RekeningTabungan)`, yang berfungsi untuk memeriksa apakah objek rekening yang sedang diproses benar-benar merupakan objek dari tipe **RekeningTabungan**, mengingat method `tambahBungaAkhirBulan()` hanya tersedia pada subclass tersebut dan tidak dimiliki oleh `Rekening` maupun `RekeningGiro`. Jika hasil pengecekan bernilai `true`, maka dilakukan **downcasting**, yaitu mengubah tipe objek tabungan (bertipe `Rekening`) menjadi tipe **RekeningTabungan** melalui `(RekeningTabungan) tabungan`, sehingga objek tersebut dapat memanggil method `tambahBungaAkhirBulan()` yang spesifik dimiliki subclass tersebut. Downcasting ini diperlukan karena secara default variabel tabungan hanya "dikenali" sebagai objek `Rekening` oleh compiler, sehingga tidak bisa

langsung mengakses method yang hanya ada di subclass tanpa proses casting terlebih dahulu.

Namun, jika objek yang diperiksa bukan merupakan RekeningTabungan (misalnya RekeningGiro), maka program akan menampilkan pesan bahwa fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan, sehingga mencegah terjadinya error `ClassCastException` yang bisa muncul apabila downcasting dipaksakan pada objek yang tipenya tidak sesuai.

Output :

Menggunakan lanjutan dari case 1 diatas :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
Pilih Menu : 7
No: 0000, Nama: Fedo
Bunga 20.0% berhasil ditambahkan : Rp40000.0
```

Jika rekening yang dipilih adalah rekening giro :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
Pilih Menu : 7
No: 0000, Nama: Fedo
Bunga 20.0% berhasil ditambahkan : Rp48000.0
No: 0000, Nama: Fedo
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan
```